Escuela Superior
de Ingeniería y Tecnología
Universidad de La Laguna

Prácticas

Programación de sockets

Redes y Sistemas Distribuidos

Grado en Ingeniería Informática 2024-2025

Iker Díaz García

(alu0101631158@ull.edu.es)

Luis Eduardo Martín Hernández

(alu0101631134@ull.edu.es)

C/ Padre Herrera s/n
38207 La Laguna
Santa Cruz de Tenerife. España

T: 900 43 25 26

ull.es



Brief Description of the developed application

This project consists of the implementation of a basic FTP server in C++ as part of the subject *Distributed Systems and Networks*. The server supports core FTP commands and allows file transfer between a client and the server over TCP sockets.

The application allows clients to: authenticate with a username and password, list server files, upload (STOR) and download (RETR) files in both active and passive modes, navigate and interact with the server through a standard FTP client.

The focus of the assignment is on managing concurrent client connections, socket programming, and implementing parts of the FTP protocol manually.

Description of the developed protocol

The server follows the basic FTP protocol structure defined by RFC 959, supporting the following commands:

USER: Accepts any username.

PASS: Validates password (**1234**).

PORT: Active mode: the server connects to the client.

PASV: Passive mode: the server listens for client connections.

STOR: Upload a file to the server.

RETR: Download a file from the server.

LIST: List directory contents.

TYPE: Accepts type specification (**TYPE I**).

QUIT: Closes the connection gracefully.

The control connection is established via a TCP socket, and a separate data connection is opened per command using either **PORT** or **PASV**.

All responses follow standard FTP status codes (e.g., 200, 150, 226, 553) as required by the protocol and test cases



Guide for the compilation of the source code and the necessary steps to execute the server program

Requirements

- Linux-based system.
- g++ compiler.
- C++ standard: C++11 or higher.

Compilation

Create a folder named build (**`mkdir build`**), and then:

```
cd build
```

```
cmake ..
```

```
make
```

Execution

Execute the file with a port: **`./ftp_server 2121`**

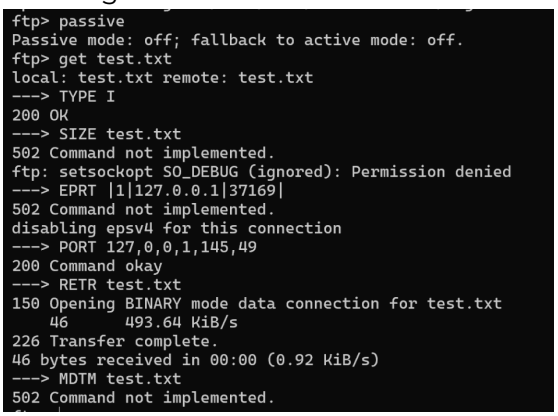
In other terminal, execute:

```
ftp -d localhost 2121
```

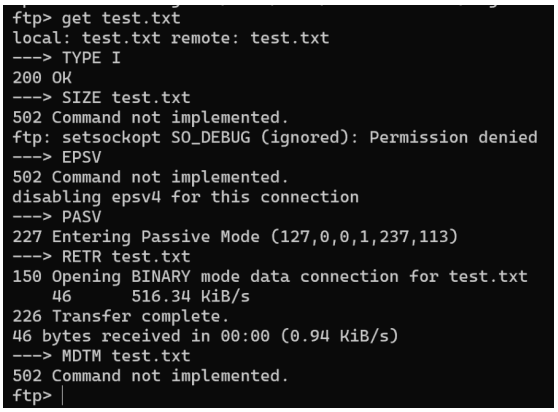
Test cases

To verify the functionality of the FTP server, we took screenshots of each test being executed:

Active get:



Passive get:



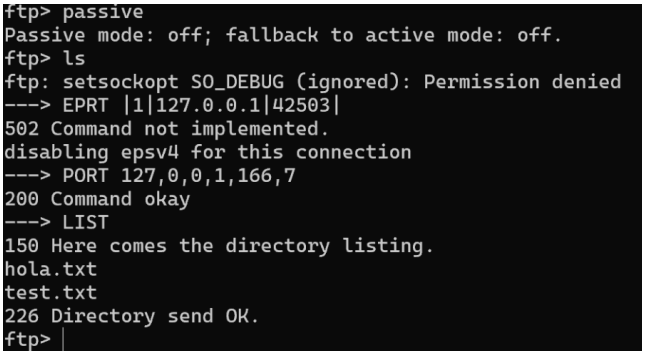
Active put:



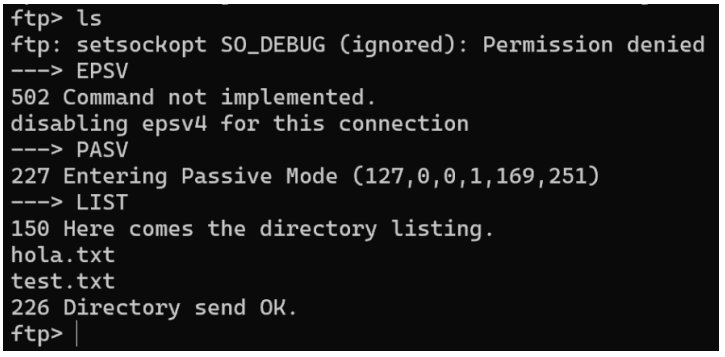
Passive put:



Active list:



Passive list:



Conclusion

This project allowed us to understand and implement key aspects of the FTP protocol from scratch using low-level socket programming in C++. By supporting both active and passive data transfer modes, and correctly handling commands such as **`PORT`**, **`PASV`**, **`STOR`**, **`RETR`**, and **`LIST`**, we gained hands-on experience with concurrent client management and protocol state handling.

Despite the complexity of handling data connections and strict compliance with expected FTP status codes, our implementation passed all functional tests and behaved correctly under different conditions. The assignment also helped us reinforce software design principles, such as modularity and error handling, while deepening our understanding of client-server communication models.

Appendix: Source code

https://github.com/Universidad-de-La-Laguna-RSD/sockets2425-pe104-9.git

References

RFC 959 - File Transfer Protocol